Sitzung 3: Generierte Forschungssoftware

Dr. Julian Dehne

Gesellschaft für Informatik e.V. (GI) · deRSE
julian.dehne@gi.de · ORCID 0000-0001-9265-9619

Einordnung

Lernziele

Nach dieser Einheit können Sie …

- einordnen, wofür Sprachmodelle im Software Engineering bisher untersucht werden,
- begründen, warum generierter Code in der Forschung andere Fragen aufwirft als in der Produktentwicklung,
- Risiken generierter Forschungssoftware benennen und einer Qualitätsdimension zuordnen,
- einschätzen, welche dieser Risiken sich im Code selbst kenntlich machen und prüfen lassen.

Ablauf

- RSE und generierter Code: Begriffe und Abgrenzung
- Stand der Forschung: Überblick, Halluzinationen, generierte Tests, Vibe Coding, Lizenzen, Leitlinien
- Weitere Befunde: Sicherheit, Abhängigkeiten, Produktivität, Reproduzierbarkeit, Code und Analysen in der Wissenschaft
- Diskussion

RSE und generierter Code

Was heißt “generierte Forschungssoftware”?

- **Generierter Code:** Quellcode, den ein Sprachmodell ganz oder teilweise erzeugt hat, von der Zeilenvervollständigung bis zum agentisch erstellten Projekt.
- **Forschungssoftware:** Software, die im Forschungsprozess oder für einen Forschungszweck entstanden ist (Gruenpeter et al. 2021).
- **Generierte Forschungssoftware:** Beides zugleich. Der Code trägt ein Forschungsergebnis, und ein Teil davon stammt nicht von der Person, die das Ergebnis verantwortet.

Stufen der Generierung

- Vervollständigung einzelner Zeilen im Editor
- Erzeugung ganzer Funktionen oder Skripte aus einer Beschreibung
- Überarbeitung bestehenden Codes (Refactoring, Übersetzung, Fehlersuche)
- Agenten, die selbstständig planen, ausführen und testen

Je höher die Stufe, desto weniger vom Code hat ein Mensch gelesen.

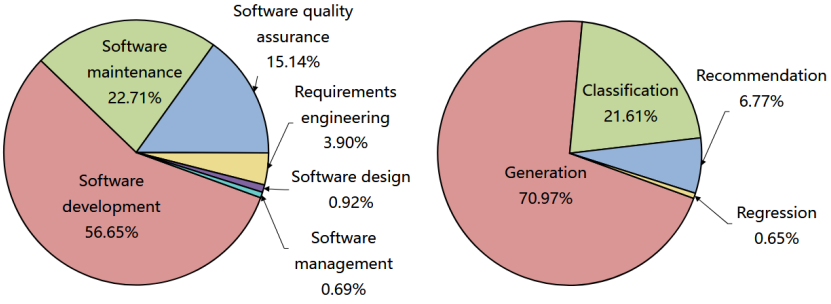
Stand der Forschung

Sprachmodelle im Software Engineering

Systematische Literaturübersicht von Hou et al. (2024):

- 395 Studien aus den Jahren 2017 bis Anfang 2024
- Vier Fragen: Welche Modelle werden eingesetzt? Mit welchen Daten? Wie werden sie angepasst und bewertet? Für welche Aufgaben?
- Guter Einstieg in das Feld, weil die Übersicht die Aufgaben nach den Phasen des Software Engineering ordnet

Wofür Sprachmodelle untersucht werden



(a) Distribution of LLM usages in SE activities. (b) Problem classification based on collected studies.

Figure 1: Verteilung der untersuchten Einsätze nach SE-Aktivität (a) und nach Problemtyp (b) (Hou et al. 2024).

Die häufigsten Aufgaben je SE-Aktivität

SE-Aktivität	Studien	Häufigste Aufgaben (Anzahl Studien)
Requirements engineering	17	Anaphoric ambiguity treatment (4), Requirements classification (4), Requirement analysis and evaluation (2), Specification generation (2)
Software design	4	GUI retrieval, Rapid prototyping, Software specification synthesis, System design (je 1)
Software development	247	Code generation (118), Code completion (22), Code summarization (21), Code search und Code translation (je 12)
Software quality assurance	66	Vulnerability detection (18), Test generation (17), Bug localization (5), Verification (5)
Software maintenance	99	Program repair (35), Code clone detection (8), Code review (7), Debugging (4)
Software management	3	Effort estimation (2), Software tool configuration (1)

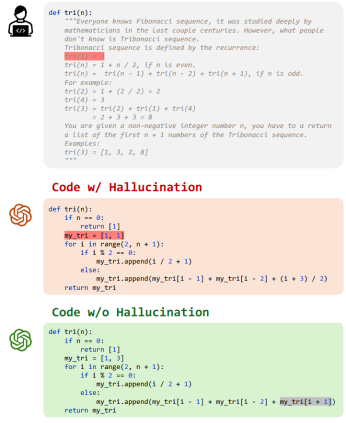
Zuordnungen nach Hou et al. (2024); eine Studie kann mehrere Aufgaben behandeln, daher 436 Zuordnungen bei 395 Studien.

Was die Verteilung zeigt

- Über die Hälfte der Studien betrifft die Softwareentwicklung (56,65 %), danach Wartung (22,71 %) und Qualitätssicherung (15,14 %).
- Anforderungen (3,90 %), Entwurf (0,92 %) und Management (0,69 %) kommen kaum vor.
- Rund 71 % der Aufgaben sind Generierungsaufgaben.

Die Forschung konzentriert sich auf das Erzeugen von Code. Die Phasen, in denen entschieden wird, *was* gebaut wird und *warum*, sind wenig untersucht.

Halluzinationen in generiertem Code



```
def tri(n):
    """Everyone knows Fibonacci sequence, it was studied deeply by
    mathematicians in the last couple centuries. However, what people
    don't know is Tribonacci sequence.
    Tribonacci sequence is defined by the recurrence:
    tri(1) = 3
    tri(n) = 1 + n / 2, if n is even.
    tri(n) =  tri(n - 1) + tri(n - 2) + tri(n + 1), if n is odd.
    For example:
    tri(2) = 1 + (2 / 2) = 2
    tri(4) = 3
    tri(3) = tri(2) + tri(1) + tri(4)
           = 2 + 3 + 3 = 8
    You are given a non-negative integer number n, you have to a return
    a list of the first n + 1 numbers of the Tribonacci sequence.
    Examples:
    tri(3) = [1, 3, 2, 8]
    """
```

Code w/ Hallucination

```
def tri(n):
    if n == 0:
        return [1]
    my_tri = [1, 3]
    for i in range(2, n + 1):
        if i % 2 == 0:
            my_tri.append(i / 2 + 1)
        else:
            my_tri.append(my_tri[i - 1] + my_tri[i - 2] + (i + 3) / 2)
    return my_tri
```

Code w/o Hallucination

```
def tri(n):
    if n == 0:
        return [1]
    my_tri = [1, 3]
    for i in range(2, n + 1):
        if i % 2 == 0:
            my_tri.append(i / 2 + 1)
        else:
            my_tri.append(my_tri[i - 1] + my_tri[i - 2] + my_tri[i + 1])
    return my_tri
```

Fig. 1. Illustration of code with or without hallucination.

Figure 2: Beispiel aus (Liu et al. 2026).

Liu et al. (2026): Was geht über funktionale Korrektheit hinaus schief?

- **Halluzination:** Die Ausgabe weicht von der Absicht ab, widerspricht sich selbst oder dem Weltwissen.
- Taxonomie aus generiertem Code: 3 Hauptkategorien, 12 Unterkategorien
- Verteilung über Modelle und Benchmarks, Ursachen und Folgen
- Gegenmaßnahmen allein über den Prompt

Im Beispiel verletzt der Code die Vorgabe `tri(1) = 3` und läuft trotzdem.

Taxonomie der Code-Halluzinationen

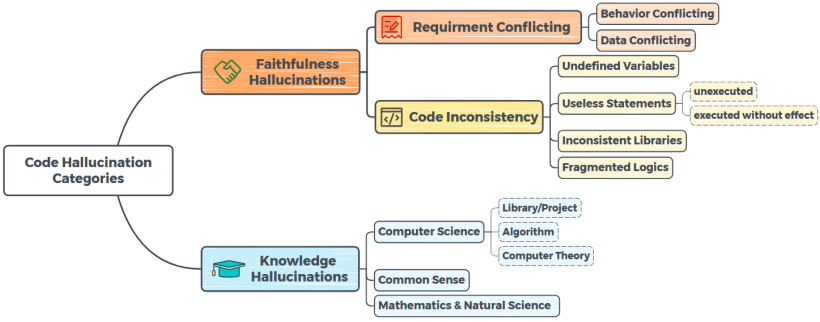


Figure 3: Kategorien von Code-Halluzinationen (Liu et al. 2026).

Generierte Unit-Tests: TestPilot

Schäfer et al. (2023): Schreiben Sprachmodelle brauchbare Unit-Tests, ohne zusätzliches Training?

- **Werkzeug:** TestPilot erzeugt Tests für alle API-Funktionen eines npm-Pakets (JavaScript).
- **Prompt:** Signatur und Implementierung der Funktion, dazu Anwendungsbeispiele aus der Dokumentation. Scheitert ein Test, wird das Modell mit dem Test und der Fehlermeldung erneut gefragt.
- **Umfang:** 25 npm-Pakete mit 1.684 API-Funktionen, Hauptmodell gpt-3.5-turbo
- **Abdeckung (Median):** 70,2 % der Anweisungen und 52,8 % der Zweige. Die feedback-gesteuerte Testgenerierung Nessie erreicht 51,3 % und 25,6 %.
- **Kein Auswendiglernen:** 92,8 % der Tests sind höchstens zu 50 % ähnlich zu vorhandenen Tests, keiner ist eine exakte Kopie.
- **Andere Modelle:** code-cushman-002 erreicht 68,2 %, StarCoder 54,0 % Anweisungsabdeckung.

Vibe Coding: Praxis, Erleben und Folgen

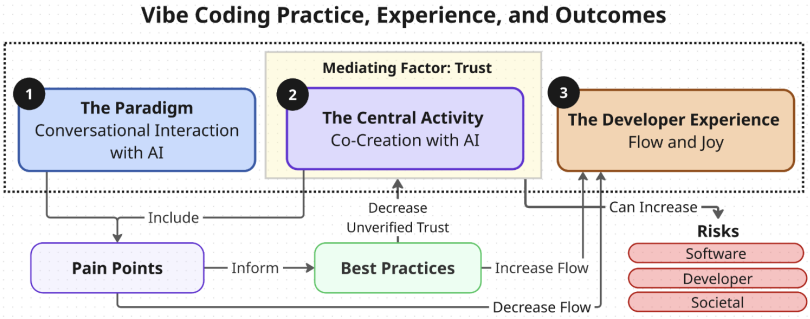


Figure 4: Vibe Coding: Praxis, Erleben und Folgen (Pimenova et al. 2026).

Vibe Coding: Befunde

Pimenova et al. (2026): Wie erleben Entwickler:innen das Programmieren im Gespräch mit der KI?

- **Daten:** über 190.000 Wörter aus halbstrukturierten Interviews, Reddit-Threads und LinkedIn-Posts, qualitativ ausgewertet
- **Theorie:** Vibe Coding verbindet die Interaktion mit der KI im Gespräch, Ko-Kreation sowie Flow und Freude beim Entwickeln.
- **Vertrauen als Regler:** Vertrauen in die KI bestimmt, wo man sich auf dem Kontinuum zwischen Delegation und Ko-Kreation bewegt, und hält den Flow aufrecht.
- **Schmerzpunkte:** Spezifikation, Zuverlässigkeit, Debugging, Latenz, Aufwand für Code-Reviews und Zusammenarbeit
- Die Studie sammelt auch Praktiken, mit denen Entwickler:innen diesen Problemen begegnen.

Generierter Code und Open-Source-Lizenzen

Xu et al. (2025) (LiCoEval, ICSE 2025): Hält generierter Code die Lizenzen seiner Vorlagen ein?

- **Herkunft:** Die Modelle sind mit lizenziertem Open-Source-Code trainiert und geben Teile davon samt Kommentaren fast wörtlich wieder.
- **Die Lizenz geht verloren:** Wiederverwendung ist erlaubt, aber an Bedingungen geknüpft, auch bei permissiven Lizenzen (z. B. Namensnennung und Änderungsvermerk bei Apache-2.0). Der generierte Code kommt ohne diese Angaben.
- **Befund:** 14 Modelle, 4.187 Python-Funktionen. Bei den drei stärksten Modellen sind 0,88 bis 2,01 % der Ausgaben einer Vorlage „auffallend ähnlich“ (*striking similarity*).
- **Nachfrage hilft kaum:** Die meisten Modelle nennen die Lizenz nicht korrekt. Bei Copyleft-Code liegt die Trefferquote bei allen Modellen bis auf eines bei null.
- **Nicht prüfbar:** Die Trainingsdaten sind meist nicht offengelegt.
- **Folge:** Das rechtliche Risiko liegt bei denen, die den Code verwenden.

Grenzen der Studie: nur Python, nur einzelne Funktionen, bewusst strenger Schwellenwert.

Zwölf Tipps für KI-gestütztes Programmieren in der Wissenschaft

Bridgeford et al. (2026): *Twelve quick tips for AI-assisted coding in science*

1. Gather background knowledge before implementation
2. Define your problem and distinguish problem framing from coding
3. Choose appropriate AI interaction models
4. Think through and divide an implementation plan
5. Manage context strategically
6. Meticulously verify AI assistance for testing
7. Use tests to specify and constrain AI-generated code
8. Leverage AI to audit and expand test coverage
9. Monitor progress and know when to restart
10. Critically review generated code
11. Refine code incrementally with focused objectives
12. Document your code for reuse and publication

Vorabversion: *Ten Simple Rules*. Die Autor:innen geben an, beim Zusammenführen der Tipps Claude genutzt zu haben.

Weitere Befunde

Sicherheit und erfundene Pakete

- **Unsicherer Code, sicheres Gefühl** (Perry et al. 2023): In einer Nutzerstudie schrieben Teilnehmende mit KI-Assistent bei vier von fünf Aufgaben häufiger unsicheren Code als ohne. Sie hielten ihren Code zugleich häufiger für sicher. Wer der KI weniger vertraute und die Prompts überarbeitete, hatte weniger Schwachstellen.
- **Erfundene Pakete** (Spracklen et al. 2025): 16 Modelle, 576.000 Codebeispiele in Python und JavaScript. Im Mittel sind mindestens 5,2 % (kommerzielle Modelle) bzw. 21,7 % (offene Modelle) der empfohlenen Pakete erfunden, insgesamt 205.474 verschiedene Namen.
- Wer einen erfundenen Namen registriert, kann darüber Schadcode verteilen. Das macht die Halluzination zum Angriff auf die Lieferkette.

Produktivität und Reproduzierbarkeit

- **Gefühlt schneller, gemessen langsamer** (Becker et al. 2025): Randomisierte Studie mit 16 erfahrenen Entwickler:innen und 246 Aufgaben in ihren eigenen, ausgereiften Open-Source-Projekten. Mit KI-Werkzeugen dauerten die Aufgaben 19 % länger. Erwartet hatten sie 24 % Zeitersparnis, nachher schätzten sie 20 %. Eine Momentaufnahme mit Werkzeugen von Anfang 2025.
- **Nicht ohne Nacharbeit lauffähig** (Vangala et al. 2025): 300 von Coding-Agenten (Claude Code, Codex, Gemini) erzeugte Projekte. Nur 68,3 % laufen ohne Nacharbeit (Python 89,2 %, Java 44,0 %). Zur Laufzeit werden im Mittel 13,5-mal so viele Abhängigkeiten gebraucht wie angegeben.

Befunde aus der Wissenschaft: Code

- **Forschungscode ist schwerer** (Tian et al. 2024): SciCode, 80 Probleme (338 Teilprobleme) aus 16 naturwissenschaftlichen Teilgebieten, von Forschenden erstellt und mit fachlichen Tests geprüft. Das beste Modell (Claude 3.5 Sonnet) löste im realistischen Szenario nur 4,6 % der Probleme.
- **Wie Forschende Sprachmodelle nutzen** (O'Brien 2025): Interviews und Interaktionsprotokolle von Wissenschaftler:innen. Sie nutzen die Modelle oft, um sich in unbekannten Sprachen und Bibliotheken zurechtzufinden, also genau dort, wo sie die Ausgabe am wenigsten prüfen können.
- **Der "AI Scientist" im Test** (Beel, Kan, and Baumgart 2025): 42 % der Experimente scheiterten an Programmierfehlern.

Befunde aus der Wissenschaft: Analysen

- **Viele KI-Analyst:innen, ein Datensatz** (Bertran, Fogliato, and Wu 2026): KI-Agenten analysieren denselben Datensatz und kommen zu deutlich unterschiedlichen Effektstärken, p-Werten und Schlussfolgerungen. Schon der Wechsel des Modells oder der Persona verschiebt die Ergebnisse, auch unter methodisch gültigen Läufen.
- **Der Garten der Pfadgabelungen, agentisch** (Miao, Pritchard, and Zou 2026): Agenten mit entgegengesetzten Vorannahmen ziehen aus denselben Daten entgegengesetzte Schlüsse. Sie reproduzieren 72 % der ideologischen Lücke aus einer Many-Analysts-Studie mit 42 Teams. 86 % dieser Analysen bestanden eine KI-Begutachtung, 78 % die Mehrheit der menschlichen Gutachter:innen.

Diskussion

Wer prüft den generierten Code?

- Wenn ein Modell den Code *und* die Tests schreibt (Schäfer et al. 2023): Woran erkennen Sie dann, dass das Ergebnis fachlich stimmt?
- Halluzinierter Code läuft oft fehlerfrei (Liu et al. 2026). Welche Prüfungen über “es läuft” hinaus braucht Ihre Analyse?
- Bridgeford et al. (2026) nutzen Tests einmal als Spezifikation (Tipp 7) und einmal als Aufgabe für die KI (Tipp 8). Wo ziehen Sie die Grenze?

Vertrauen und Selbsteinschätzung

- Gefühlt schneller, gemessen langsamer (Becker et al. 2025); gefühlt sicher, tatsächlich unsicherer (Perry et al. 2023). Wie überprüfen Sie Ihr eigenes Urteil?
- Vertrauen hält den Flow aufrecht (Pimenova et al. 2026). Wann wird Flow in der Forschung zum Risiko?
- Würden Sie generierten Code so reviewen wie den Code einer Kollegin? Was hält Sie davon ab?

Reproduzierbarkeit und Herkunft

- Erfundene Pakete, fehlende Abhängigkeiten (Spracklen et al. 2025; Vangala et al. 2025): Was muss zu generiertem Code dazugehören, damit Dritte ihn ausführen können?
- Die Lizenz der Vorlage geht verloren (Xu et al. 2025): Wer trägt das Risiko, wenn Ihre Publikation fremden Code ohne Lizenzangabe enthält?
- Analysen mit Agenten streuen je nach Modell und Vorannahme (Bertran, Fogliato, and Wu 2026; Miao, Pritchard, and Zou 2026). Was müssen Sie offenlegen: Code, Prompts, Agentenverlauf, verworfene Analysen?
- Das Modell von heute ist morgen nicht mehr verfügbar. Genügt es, den Code zu archivieren, oder gehören Prompts und Modellversion dazu?

Verantwortung und Offenlegung

- Bridgeford et al. (2026) legen ihre KI-Nutzung offen. Wie sollte eine Publikation den Anteil generierten Codes kenntlich machen?
- Wer verantwortet die Fehler generierter Forschungssoftware: wer promptet, wer publiziert, wer das Modell anbietet?
- Die Forschung untersucht vor allem das Erzeugen von Code (Hou et al. 2024). Welche Fragen bleiben für Forschungssoftware offen?
- Bezug zu `rse_annotations`: Welche dieser Risiken ließen sich als Concern im Code markieren und prüfen?

Literatur I

Becker, Joel, Nate Rush, Elizabeth Barnes, and David Rein. 2025. “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity.” https://doi.org/10.48550/arXiv.2507.09089.

Beel, Joeran, Min-Yen Kan, and Moritz Baumgart. 2025. “Evaluating Sakana’s AI Scientist: Bold Claims, Mixed Results, and a Promising Future?” https://doi.org/10.48550/arXiv.2502.14297.

Bertran, Martin, Riccardo Fogliato, and Zhiwei Steven Wu. 2026. “Many AI Analysts, One Dataset: Navigating the Agentic Data Science Multiverse.” https://doi.org/10.48550/arXiv.2602.18710.

Bridgeford, Eric W., Iain Declan Campbell, Zijiao Chen, Zhicheng Lin, Harrison Ritz, Joachim Vandekerckhove, and Russell A. Poldrack. 2026. “Twelve Quick Tips for AI-Assisted Coding in Science.” *PLOS Computational Biology* 22 (7): e1014428. https://doi.org/10.1371/journal.pcbi.1014428.

Gruenpeter, Morane, Daniel S. Katz, Anna-Lena Lamprecht, et al. 2021. “Defining Research Software: A Controversial Discussion.” Zenodo. https://doi.org/10.5281/zenodo.5504016.

Literatur II

Hou, Xinyi, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. “Large Language Models for Software Engineering: A Systematic Literature Review.” *ACM Transactions on Software Engineering and Methodology* 33 (8): 1–79. https://doi.org/10.1145/3695988.

Liu, Fang, Yang Liu, Lin Shi, Zhen Yang, Li Zhang, Xiaoli Lian, Zhongqi Li, and Yuchi Ma. 2026. “Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code.” https://doi.org/10.48550/arXiv.2404.00971.

Miao, Jiacheng, Jonathan K. Pritchard, and James Zou. 2026. “The Agentic Garden of Forking Paths.” https://doi.org/10.48550/arXiv.2607.01507.

O’Brien, Gabrielle. 2025. “How Scientists Use Large Language Models to Program.” In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. https://doi.org/10.1145/3706598.3713668.

Perry, Neil, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. “Do Users Write More Insecure Code with AI Assistants?” In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. https://doi.org/10.1145/3576915.3623157.

Literatur III

Pimenova, Veronica, Sarah Fakhoury, Christian Bird, Margaret-Anne Storey, and Madeline Endres. 2026. “Good Vibrations? A Qualitative Study of Co-Creation, Communication, Flow, and Trust in Vibe Coding.” https://doi.org/10.48550/arXiv.2509.12491.

Schäfer, Max, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. “An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation.” https://doi.org/10.48550/arXiv.2302.06527.

Spracklen, Joseph, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. 2025. “We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs.” In *34th USENIX Security Symposium*, 3687–3706. https://arxiv.org/abs/2406.10279.

Tian, Minyang et al. 2024. “SciCode: A Research Coding Benchmark Curated by Scientists.” https://doi.org/10.48550/arXiv.2407.13168.

Vangala, Bhanu Prakash, Ali Adibifar, Ashish Gehani, and Tanu Malik. 2025. “AI-Generated Code Is Not Reproducible (yet): An Empirical Study of Dependency Gaps in LLM-Based Coding Agents.” https://doi.org/10.48550/arXiv.2512.22387.

Literatur IV

Xu, Weiwei, Kai Gao, Hao He, and Minghui Zhou. 2025. “LiCoEval: Evaluating LLMs on License Compliance in Code Generation.” In *Proceedings of the 47th International Conference on Software Engineering (ICSE 2025)*. https://doi.org/10.48550/arXiv.2408.02487.